

System Log Anomaly Detection With Noise-Contrastive Learning and Pattern Feature

Pengcheng Luo, Dengke Deng, Mingfeng Xie[✉], Genke Yang[✉], Jian Chu, Boon-Hee Soong[✉], *Senior Member, IEEE*, and Chau Yuen[✉], *Fellow, IEEE*

Abstract—System logs play a critical role in identifying security threats such as network attacks, unauthorized access, and system vulnerabilities. Recent research has focused on extracting sequences of security-related events from large-scale log data and applying deep learning methods for anomaly detection. These methods achieve anomaly detection by designing an auxiliary training task that predicts the next event. However, the inconsistency between the task of event prediction and the goal of anomaly detection limits their performance. In this paper, we introduce a noise-contrastive learning approach that introduces synthetic noise as a training signal, enabling the model to directly discriminate between normal and anomalous samples. Our approach also includes an ensemble model, STrees, and a novel feature engineering method named pattern feature. By modeling the relationships among events under normal conditions, these methods allow the model to capture contextual dependencies in log event sequences, thereby improving its ability to detect potential anomalies. Specifically, using the pattern feature improves anomaly-class recall by at least 7.3% compared to using event sequences as input. Experimental results demonstrate that our model improves anomaly-class recall by at least 5.3% over advanced log anomaly detection methods while achieving more accurate detection in most comparisons across datasets. These results highlight the effectiveness of our approach in learning discriminative decision boundaries between normal and anomalous event sequence patterns.

Index Terms—System log, anomaly detection, contrastive learning, synthetic data, machine learning.

I. INTRODUCTION

A. Background and Motivation

WITH the growing complexity of cyber threats, cybersecurity has become a critical global concern. As attackers

adopt increasingly advanced techniques, the effectiveness of traditional security systems in protecting digital assets is being significantly undermined [1], [2]. System logs provide detailed records of service activities, system states, user operations, and external access events [3], [4], making them a valuable resource for security analysis. Analyzing these logs plays a vital role in identifying malicious activities, abnormal behaviors, and potential system vulnerabilities.

In a typical enterprise information technology (IT) environment, a Security Information and Event Management (SIEM) system processes a substantial volume of logs generated by Host-based Intrusion Detection Systems (HIDS) and Network-based Intrusion Detection Systems (NIDS) [5]. The SIEM analyzes these logs to detect potential anomalies within the network service and servers, determining whether alerts should be generated [6]. Specifically, upon receiving the logs, the SIEM parses the data, extracts events sequentially based on predefined templates, and constructs corresponding event sequences. Subsequently, anomaly detection models are applied to the event sequences to identify anomalies. The system log analysis workflow is depicted in Fig. 1.

The accuracy and robustness of this workflow are inherently dependent on the structure and consistency of the underlying system log data. System logs are generated by the printing instructions in the source code of the network service programs. These logs convey the program's runtime status and document user operations [7], [8]. A system log comprises a series of ordered log statements, each of which asserts the occurrence of specific events within the system [9]. These log statements are grouped based on certain criteria, such as unique identifiers (UIDs) or time windows. Such groupings are referred to as “log segments.” Each log statement within a log segment represents at least one event, such as “user login successful.” By parsing the statements within a log segment, the corresponding event sequence can be extracted.

Anomaly detection aims to model the normal behavior of a system by analyzing event sequences collected during its regular operation [10]. Subsequent significant deviations from the learned patterns are identified as anomalies [11]. For instance, in a network security service context, a normal event sequence might include actions such as “host boot successful,” followed by “security policies loaded,” “firewall rules updated,” and “network traffic monitoring initiated.” If an event sequence contains

Received 23 January 2025; revised 8 June 2025; accepted 10 June 2025. Date of publication 16 June 2025; date of current version 21 November 2025. This work supported in part by the National Key Research and Development Program of China under Grant 2020YFB1711200, in part by China Scholarship Council Program under Project 202406230244, and in part by the Ministry of Education of Singapore under Grant RG62/22. Recommended for acceptance by Prof. Yuanwei Liu. (Corresponding authors: Genke Yang; Chau Yuen.)

Pengcheng Luo, Dengke Deng, Genke Yang, and Jian Chu are with Ningbo Artificial Intelligence Institute, Shanghai Jiao Tong University, Ningbo 315012, China, and also with the Department of Automation, Shanghai Jiao Tong University, Shanghai 200240, China (e-mail: luopeng69131@sjtu.edu.cn; pokier123@sjtu.edu.cn; gkyang@sjtu.edu.cn; chujian@niii.com).

Mingfeng Xie is with the College of Electronic and Information Engineering, Nanjing University of Aeronautics and Astronautics, Nanjing 210016, China (e-mail: xiemi1997@nuaa.edu.cn).

Boon-Hee Soong and Chau Yuen are with the School of Electrical and Electronics Engineering, Nanyang Technological University, Singapore 639798 (e-mail: ebhsoong@ntu.edu.sg; yuenchau@gmail.com).

Digital Object Identifier 10.1109/TNSE.2025.3579809

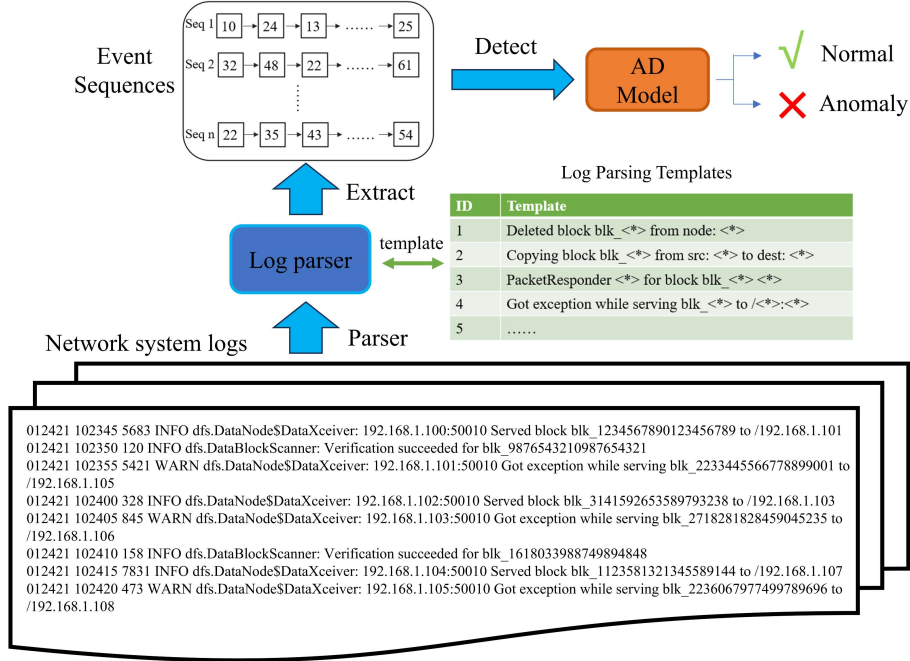


Fig. 1. The system log processing workflow.

“host boot successful” and “network traffic monitoring initiated” but omits critical intermediate steps, such as “security policies loaded” or “firewall rules updated,” it could be identified as anomalous. Such anomalies may signal potential configuration errors, tampering with security policies, or unauthorized system access [12].

Deep learning (DL) has become a powerful tool for anomaly detection, offering substantial improvements over classical methods through its strong representation learning capabilities. DL has driven the development of general-purpose deep anomaly detection (DeepAD) techniques, such as DeepSVDD [13] and Deep Isolation Forest (DIF) [14]. In parallel, DL has also enabled significant progress in log anomaly detection, leading to the development of log-structured deep learning models (LogDeep). These methods typically leverage Long Short-Term Memory (LSTM) networks [13], [15], [16], [17] to predict the next event in a log event sequence. If the predicted event deviates from the actual one, an anomaly is flagged, indicating potential abnormal behavior in the system.

Despite these advancements, system log anomaly detection still faces several key challenges. Firstly, existing methods [18], [19] train anomaly detection models by designing an intermediate event prediction task, which leads to inconsistency between the task of event prediction and the ultimate goal of anomaly detection. Secondly, they fail to fully leverage contextual dependencies and model inter-event relationships to effectively detect anomalies within event sequences [16], [17]. Lastly, existing DL-based anomaly detection methods are not CPU-friendly, and the high computational cost of GPU inference makes them unsuitable for practical use in enterprise network departments.

B. Contributions

To address the aforementioned challenges, we propose the Noise-Contrastive Learning (NCL) method. It enables the model

to directly learn to determine whether an event sequence contains anomalies, thereby avoiding inconsistencies between event prediction training and the anomaly detection task. We propose pattern features, which model inter-event relationships and indicate the potential anomaly status of each event in a log sequence, thereby enhancing the model’s ability to detect contextual anomalies. Additionally, we utilize an ensemble machine learning (ML) model based on decision trees. It makes the model more CPU-friendly. The key contributions of our research are as follows:

- We propose the NCL method, which formulates log anomaly detection as a binary classification task by introducing synthetic noise as negative samples, enabling the model to perform direct anomaly detection without relying on auxiliary prediction tasks.
- We introduce STrees, an ensemble learning model based on decision trees, and a feature engineering method called pattern feature. These methods effectively capture contextual dependencies in event sequences, enabling the model to learn discriminative decision boundaries.
- We conduct extensive testing across various real-world system log datasets, including HDFS, BGL, and Hadoop, to evaluate the performance of our model. Experimental results demonstrate that for anomaly detection tasks, our model achieves more accurate detection in most comparisons against LogDeep methods and exhibits more balanced detection performance compared to DeepAD methods.

C. Organizations

The paper is structured as follows: Section II reviews the literature on anomaly detection in system logs. In Section III, we introduce our proposed pattern feature, the NCL approach, and

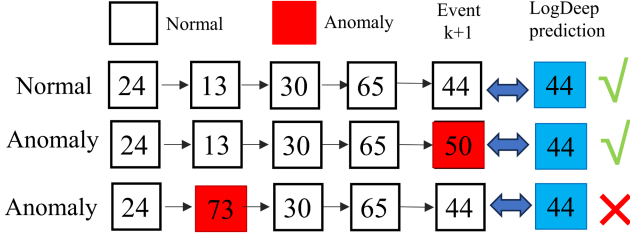


Fig. 2. Limitation of next-event prediction methods in detecting contextual anomalies.

the STrees model. Section IV assesses our model’s performance on real-world system log datasets. Finally, Section V concludes the research and outlines avenues for future work.

II. RELATED WORK

System log anomaly detection plays a critical role in maintaining the reliability, security, and operational efficiency of computing systems. Recent years have witnessed a proliferation of DL-based methods for this task. These methods can be broadly divided into two categories: (i) log-structured deep learning anomaly detection approaches, which explicitly utilize the semantic and sequential event of log data; and (ii) general-purpose deep anomaly detection algorithms, which are originating from other domains such as vision or tabular anomaly detection.

A. LogDeep: Log-Structured Deep Learning Anomaly Detection Methods

LogDeep methods are specifically engineered to handle the unique characteristics of system logs, typically involving stages like log parsing, event sequence construction, and anomaly detection. These approaches aim to learn the patterns of normal log sequences and detect anomalies as deviations from these learned patterns. Typical training objectives include next-event prediction and masked event modeling.

1) *Next Event Prediction-Based Methods:* DeepLog [20] introduced a universal DL framework. It first employs a log parser [21] to extract log keys from unstructured logs, which are then treated as events. These events are formed into chronological sequences, and an LSTM network is trained to model normal event sequences by predicting the next log event. Anomalies are flagged when the actual log event deviates significantly from the LSTM’s prediction. Building on this framework, Meng et al. [17] proposed LogAnomaly, which enhances semantic representation through template2vec and jointly models both sequential dependencies and statistical patterns in event occurrences, enabling more comprehensive anomaly detection. While effective in capturing sequential regularities, a key limitation of these event prediction-based methods lies in their inherent focus on forecasting, which can make it challenging to identify subtle or contextual anomalies embedded within an event sequence, as opposed to those occurring as unexpected next events. Fig. 2 provides an example of a contextual anomaly that escapes detection under the next-event prediction paradigm.

2) *Contextual Embedding and Transformer-Based Methods:* Recent approaches employ contextual embeddings and Transformer [22] architectures to capture richer dependencies. LogBERT [18] adopts a Bidirectional Encoder Representations from Transformers (BERT)-style encoder [23] with masked event prediction and self-supervised volume minimization for training. LAnoBERT [19] further refines this approach by customizing loss functions per log key and improves detection accuracy via more structured training. Additionally, Zhang et al. [24] propose an attention-based bidirectional LSTM that learns discriminative features across log sequences. These models offer better generalization and leverage long-range dependencies, but often depend on auxiliary training tasks, which may not be directly aligned with anomaly detection objectives.

However, despite these advancements in capturing context, many of these LogDeep methods still commonly rely on an intermediate auxiliary prediction task (e.g., next event prediction, masked event prediction) to train the anomaly detection models. While this pre-training or self-supervision can help in learning robust log representations, the anomaly detection capability itself is often an indirect outcome of optimizing for these proxy tasks. This indirect approach may not always be the most direct or optimal way to learn discriminative features that specifically distinguish anomalous log sequences from normal ones.

B. DeepAD: General-Purpose Deep Anomaly Detection Algorithms

DeepAD methods are domain-agnostic algorithms originally developed for anomaly detection in vision, medical, or tabular data. Although not tailored to log data, DeepAD methods offer strong representation learning capabilities. In particular, their self-supervised learning paradigms show promising potential for adaptation to log anomaly detection tasks. Representative DeepAD techniques can be categorized into the following three groups.

1) *Representation-Based One-Class Models:* DeepSVDD [13] extends the classical Support Vector Data Description (SVDD) [25] framework by training a neural network to map normal samples into a compact hypersphere in latent space. Anomalies are identified by their deviation from this center. However, the model depends on high-quality feature representations and does not capture temporal dependencies, which are crucial in sequential data like system logs.

2) *Transformation-Based Self-Supervised Methods:* Transformation-based self-supervised methods offer an alternative perspective on learning discriminative representations without labeled anomalies. GOAD [26] applies random affine transformations and trains models to classify them, enabling unsupervised detection of out-of-distribution samples. NeuTraL [27] jointly learns transformations and representations to better distinguish normal and anomalous instances. These methods are robust and flexible but assume spatial structure or visual patterns in the data, which may not transfer effectively to text-based log sequences.

TABLE I
COMPARISON BETWEEN LOGDEEP AND DEEPAD METHODS

Aspect	LogDeep	DeepAD
Utilizes log structure	Yes	No
Temporal modeling	Strong (LSTM/BERT)	Limited or none
Training target	Sequence prediction / masking	Representation optimization
Domain specificity	High (log-specific)	General-purpose
context-awareness	Sequence-level	Instance-level
Direct anomaly scoring	Limited (proxy tasks)	Often yes

TABLE II
A COMPARATIVE ANALYSIS OF TECHNIQUES AND LIMITATIONS IN UNSUPERVISED ANOMALY DETECTION METHODS

Method	Class	Technical Aspects	Notable Limitations
DeepLog [20]	LogDeep	LSTM, log templates, predict next event	limited contextual representation ability
LogAnomaly [17]	LogDeep	LSTM, template2vec semantic features, predict next event	Struggles with long sequences
DeepCase [16]	LogDeep	LSTM, contextual analysis, cluster, predict next event	Struggles with uncertain prediction
LogBERT [18]	LogDeep	BERT, log templates, predict masked event, sphere loss	insensitive to complex anomaly pattern
LAnoBERT [19]	LogDeep	BERT, predict masked event, one-class, tokenizer for parser	struggles to train and converge
DeepSVDD [13]	DeepAD	deep feature representation, compact sphere loss, one-class model	limits to capture intricate semantic patterns
GOAD [26]	DeepAD	transformation-based learning, open-set classification, distance-based anomaly scoring	limited by high-dimensional features
ICL [29]	DeepAD	contrastive learning, self-supervised masking, consecutive feature subsets	dependency assumptions may not hold universally
NeuTraL [27]	DeepAD	learnable transformations, deterministic contrastive loss, integrated anomaly scoring	limited by large-scale datasets
RCA [30]	DeepAD	collaborative training, sample selection mechanism, robustness to noisy data	limited by structured or highly contextual datasets
RDP [31]	DeepAD	unsupervised representation learning, distance preservation, task flexibility	relies on random projections
REPen [32]	DeepAD	customized representation learning, triplet sampling, integration of labeled outliers	limits to capture complex feature relationships
DIF [14]	DeepAD	random representation ensemble, synergy of representations and isolation, deviation-enhanced anomaly scoring	relies on the quality of random initializations
STrees (ours)	/	decision tree, context-aware, contrastive learning, discriminative model	Not fully distilled the ensemble model

3) *Forest-Based Ensemble Methods*: Forest-based approaches such as DIF [14] enhance the traditional Isolation Forest (IF) [28] by integrating random deep representation learning and downstream scoring mechanisms. DIF demonstrates superior performance on high-dimensional and non-linearly separable data and maintains scalability. However, it does not model temporal or semantic dependencies in log sequences.

Although DeepAD models are not designed specifically for log data, some of their core ideas have been adopted in LogDeep approaches [18], [19]. For instance, LogBERT integrates a hypersphere volume minimization objective inspired by DeepSVDD [13], adapting it to a self-supervised learning setting for modeling normal log sequences. However, the DeepAD methods generally lack the ability to exploit the contextual dependencies inherent in log data, which may lead to suboptimal performance in log-specific scenarios.

C. Summary and Research Gaps

Table I presents a comprehensive comparison between the LogDeep and the DeepAD methods. LogDeep methods benefit from tight coupling with log semantics but suffer from over-reliance on preprocessing and indirect training objectives. DeepAD methods offer flexible and theoretically grounded alternatives, but they often lack integration with the temporal

and contextual structural properties of log data. A key research gap remains in developing models that simultaneously capture contextual dependencies and support direct anomaly detection.

Our work addresses this gap by proposing a discriminative, context-aware anomaly detection model that directly identifies anomalies within log event sequences, without relying on auxiliary prediction tasks. This approach enhances generalization while retaining the advantages of log-specific structural modeling. Table II presents a comparative overview of representative methods, highlighting their key technical characteristics and inherent limitations.

III. PROPOSED LOG ANOMALY DETECTION FRAMEWORK

Given an event sequence $\mathbf{x} = (k_1, k_2, \dots, k_n)$ extracted from system logs, the goal of the log anomaly detection task is to obtain a model g that can identify whether the event sequence $\mathbf{x}$ is an anomalous sample. Each event sequence $\mathbf{x}$ is associated with a ground-truth label $y \in \{0, 1\}$, where 0 denotes an anomaly and 1 denotes a normal case. The model g aims to predict the label $\hat{y}$:

$$\hat{y} = g(\mathbf{x}), \quad (1)$$

where $\hat{y} \in \{0, 1\}$ is the predicted label.

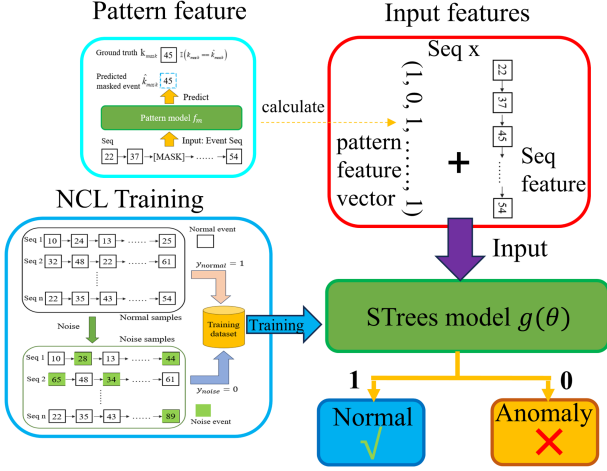


Fig. 3. Proposed anomaly detection model pipeline.

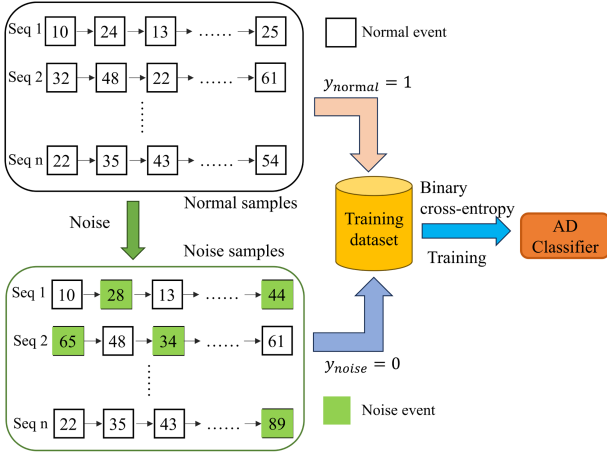


Fig. 4. Noise-contrastive learning framework.

To detect anomalies in event sequences, we propose a log anomaly detection framework centered on two key ideas: direct anomaly detection training and the modeling of contextual dependencies. First, we introduce the NCL method, which formulates anomaly detection as a binary classification task by constructing synthetic noise samples as negative examples, removing the need for auxiliary prediction objectives. Then, to improve the model's ability to capture inter-event relationships and contextual dependencies, we propose STrees, a decision tree-based ensemble model, together with a novel input feature termed pattern feature. These components together enable effective learning of discriminative boundaries in feature space. The overall architecture of our proposed solution is illustrated in Fig. 3.

A. Noise-Contrastive Learning

Anomaly detection is inherently challenging due to the rarity and diversity of anomalies, which makes them difficult to define and collect. This challenge is particularly pronounced in log event anomaly detection, where labeled anomalous sequences are scarce and often ill-defined. As a result, most models are trained exclusively on normal samples.

In the case of log data, the structured and procedural nature of system logs enables a cost-effective strategy for synthesizing surrogate anomaly samples. Normal log sequences are typically generated through a series of well-defined and repeatable operational routines executed by system programs. When extraneous noise events are injected into these sequences, they interrupt the expected control flow, and the resulting deviations can simulate abnormal behavior. Although such synthetic disturbances do not exactly replicate real anomalies, they provide effective negative training samples for discriminative models. By learning to distinguish these noise-injected sequences from normal ones, the models are able to identify boundaries between normal and abnormal patterns. This approach allows the model to generalize to subtle deviations, even when real anomaly labels are unavailable.

To formalize this idea within a principled learning framework, we draw inspiration from Noise-Contrastive Estimation [33] and propose the Noise-Contrastive Learning (NCL) method for log anomaly detection. In the NCL framework, three distinct distributions are considered. The first, denoted by $p_r(x)$, represents the distribution of normal data. The second distribution, $p_a(x)$, corresponds to genuine anomalous data. The third, $q(x)$, is a noise distribution introduced to provide negative samples for contrastive learning. Unlike true anomalies, which are rare and often ill-defined, the noise distribution is artificially constructed to supply negative samples in place of real anomalies. These noise samples, combined with normal samples, are used to train the anomaly detection classifier to establish a discriminative decision boundary. To determine whether a given sample x is a or non-normal, a discriminator g_θ is introduced.

The objective of the NCL framework is to optimize the parameter θ by training the discriminator g_θ to distinguish between normal and non-normal samples. Here, $y_n \in \{1, 0\}$ represents whether a sample x is a normal or noise sample. The posterior probability that sample x comes from the normal data distribution is:

$$p(y_n = 1 | x) = g_\theta(x). \quad (2)$$

In contrast, the posterior probability that sample x comes from the noise distribution is: $p(y_n = 0 | x) = 1 - p(y_n = 1 | x)$.

Sampling I samples, $x_1, x_2, \dots, x_I$, from the normal distribution $p_r(x)$, assigning their labels as 1. Then sampling KI samples, $x'_1, x'_2, \dots, x'_{KI}$, from the noise distribution $q(x)$, assigning their labels as 0. The number of noise samples is K times the number of normal samples. The objective of NCL is to distinguish between normal and noise samples, which is a binary classification problem. The loss function of NCL is:

$$L(\theta) = -\frac{1}{I(K+1)} \left(\sum_{i=1}^I \log p(y_n = 1 | x_i) + \sum_{i=1}^{KI} \log p(y_n = 0 | x'_i) \right). \quad (3)$$

The generation of noise samples is shown in Algorithm 1, which aims to provide negative samples for NCL training. The function `random_noise(x,e)` randomly selects e events from x

Algorithm 1: Generation of Noise Sample Dataset.

```

1: Input: Normal sequence dataset
    $D_{normal} = \{(x_i, y_i)\}_{i=1}^I$ , the number of noise
   events  $e$ 
2: Initialize noise dataset  $D_{noise}$  to  $\emptyset$ 
3: for each  $x$  in  $D_{normal}$  do
4:   for  $k = 1$  to  $K$  do
5:      $x_{noise} = \text{random\_noise}(x, e)$ 
6:      $D_{noise} = D_{noise} \cup \{x_{noise}\}$ 
7:   end for
8: end for
9: Output: Noise dataset  $D_{noise}$ 

```

and replaces them with new events drawn from the entire event vocabulary of the logs. The parameter e , representing the number of events replaced, is a key hyperparameter that controls the similarity between noise samples and normal samples. This similarity influences the tightness of the decision boundary learned by the anomaly detection classifier.

After incorporating noise data as negative samples, we construct a dataset by combining them with normal event sequences. This dataset is then used to train an anomaly detection classifier, optimized using the binary cross-entropy loss defined in (3). This allows the model to directly determine whether an event sequence is normal or anomalous. The complete NCL training process is shown in Fig. 4.

During NCL training, the anomaly detection classifier is trained on normal samples labeled as positive and synthesized noise samples labeled as negative, without access to real anomalous data. The goal of NCL is to drive the classifier to learn a decision boundary that effectively separates normal patterns from artificially introduced perturbations. Intuitively, this can be understood as learning a boundary in the feature space that tightly “encloses” the region of normal sample distributions, thereby distinguishing normal samples from both the noise samples outside the boundary and unseen real anomalies. The use of random replacement to generate noise samples simulates disturbances in normal sequences. These disturbances force the model to learn and recognize the inherent patterns of normal sequences, thus forming a well-defined and discriminative classification boundary.

The number of noise events, e , determines how similar the noise samples are to normal samples. This similarity, in turn, affects the tightness of the learned decision boundary. When e is small, the noise samples differ only slightly from the normal ones. To distinguish between them, the classifier must learn a tighter decision boundary that closely conforms to the distribution of normal samples. This tighter boundary makes the model more sensitive to subtle deviations, which theoretically improves the detection of minor anomalies that do not deviate significantly from normal patterns. When e is large, the noise samples differ significantly from the normal ones. The classifier can then learn a relatively looser decision boundary, which is farther from the distribution of normal samples. Such a looser boundary allows for greater tolerance of variations within normal samples and

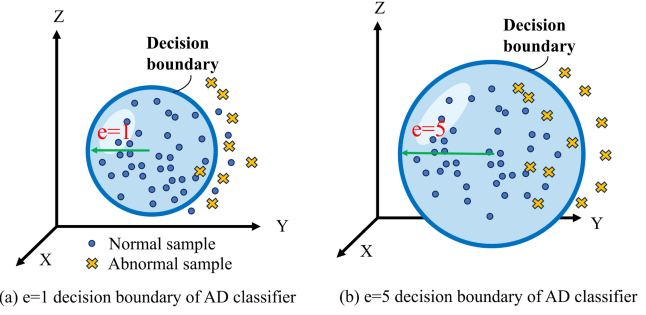


Fig. 5. Tight (left) and loose (right) decision boundaries of the anomaly detection classifier trained by NCL.

may reduce false alarms, but it might also lower the model sensitivity to less obvious anomalies. The effect of different numbers of noise events on the decision boundary is shown in Fig. 5.

To summarize, the parameter e affects the model’s sensitivity to anomalies by shaping the decision boundary. Smaller values of e improve detection of subtle anomalies but may increase false alarms, while larger values reduce false alarms at the risk of missing less obvious anomalies. Thus, tuning e provides a practical way to balance false positives and false negatives. The value of e is determined through experimental evaluation.

B. STrees: An Ensemble Model Based on Decision Trees

We propose STrees, a fully tree-based ensemble model inspired by the Gradient Boosting Decision Tree (GBDT) + Logistic Regression (LR) framework [34], which effectively captures feature interactions and supports large-scale classification. STrees integrates Feature Representation Decision Trees (FRDT) as a feature extractor and Extremely Randomized Trees (ExTree) [35] as the final classifier. By adopting a fully tree-based architecture, STrees efficiently leverages CPU computation while capturing inter-event relationships within event sequences. Compared to Random Forests [36] and XGBoost [37], STrees separates feature representation and classification, enabling more robust modeling on high-dimensional, sparse features with stronger noise tolerance and reduced risk of overfitting.

To build STrees, we first apply FRDT to transform input data into high-dimensional sparse feature representations. These features are then used to train an ExTree classifier. The steps for training and prediction with STrees are presented in Algorithm 2. Specifically, lines 5-10 correspond to the feature extraction by FRDT, while lines 12-13 represent the training of the ExTree classifier. Lines 15-18 illustrate how the trained STrees model is used for prediction on new samples.

The FRDT model contains multiple decision trees, and each decision tree is similar to a filter in a Convolutional Neural Network (CNN). For training the FRDT model, the Light Gradient Boosting Machine (LightGBM) algorithm [38] is chosen as the training algorithm. In the training process, the decision tree at iteration t sequentially corrects the mistakes of their predecessors by focusing on the residual errors indicated by

Algorithm 2: STrees: Training and Prediction.

```

1: Input: Training dataset  $D = \{(x_i, y_i)\}_{i=1}^I$ 
2: Step 1: Train FRDT Model
3: Train FRDT model using Algorithm 3
4: Step 2: Feature Transformation Using FRDT Model
5: for each sample  $x_i$  in the training dataset  $D$  do
6:   for each tree  $F_t$  in the FRDT model  $f_T$  do
7:      $s_t = \text{get\_leaf\_index}(F_t, x_i)$ 
8:   end for
9:    $x'_i = (s_1, s_2, \dots, s_n)$ 
10: end for
11: The transformed dataset is  $D' = \{(x'_i, y_i)\}_{i=1}^I$ .
12: Step 3: Train ExTree Model
13: Use the transformed dataset  $D'$  to train an ExTree model.
14: Step 4: Prediction
15: for each new sample  $x_{\text{new}}$  do
16:   Transform  $x_{\text{new}}$  using the trained FRDT model to get  $x'_{\text{new}}$ .
17:   Use the trained ExTree model to predict the label of  $x'_{\text{new}}$ .
18: end for
19: Output: Trained STrees model FRDT+ExTree

```

negative gradients. This stepwise refinement aims to reduce prediction errors in each iteration, thus improving the overall fitting ability of the model [39]. The negative gradient for the i -th sample, x_i , in the t -th round is defined as follows:

$$r_{ti} = - \left[\frac{\partial L(y_i, f(x_i))}{\partial f(x_i)} \right]_{f(x)=f_{t-1}(x)}, \quad (4)$$

where $f_{t-1}(x)$ is the FRDT model after the $t-1$ th round of the iteration. The training process of the FRDT is presented in Algorithm 3. The sum of the gradients and Hessians of the left subtree and the right subtree is G_L , G_R , H_L , and H_R .

In the second stage of the STrees model, we employ ExTree [35] as the final classifier. The first-stage model FRDT outputs the indices of the leaf nodes into which each sample falls across the ensemble of base decision trees. These indices form a sparse and high-dimensional feature representation. Previous research has demonstrated that ExTree is suited for high-dimensional data, offering both strong predictive performance and computational efficiency [40]. Unlike standard decision trees, ExTree does not seek the optimal split threshold for each feature. Instead, it randomly selects thresholds for evaluation. This approach significantly reduces computational cost due to its inherent randomness and acts as an effective form of regularization. It substantially lowers model variance and mitigates overfitting, particularly in the context of complex and nonlinearly separable datasets.

C. Pattern Feature and Fusion Feature

Previous studies have shown that anomalies can be effectively detected by identifying deviations from modeled patterns of

Algorithm 3: Gradient Boosting Decision Tree Algorithm.

```

1: Input: Training data  $D = \{(x_i, y_i)\}_{i=1}^I$ , the number of boosting rounds  $T$ , learning rate  $\eta$ , loss function  $l$ , regularization parameters  $\lambda$  and  $\gamma$ 
2: Initialize model with a constant value  $\omega$ :  $f_0(x) = \arg \min_{\omega} \sum_i l(y_i, \omega)$ 
3: for  $t = 1$  to  $T$  do
4:   Compute gradients and Hessians:
5:   for each  $x_i$  in  $D$  do
6:      $g_{ti} = \partial_{y_i} l(y_i, f_{t-1}(x_i))$ 
7:      $h_{ti} = \partial_{y_i}^2 l(y_i, f_{t-1}(x_i))$ 
8:   end for
9:   Set  $G_t = \sum_{i=1}^I g_{ti}$ ,  $H_t = \sum_{i=1}^I h_{ti}$ 
10:  Construct a tree  $F_t$  by recursively partitioning the data  $D$  to maximize the gain in objective:
     $gain = \frac{1}{2} \frac{G_L^2}{H_L + \lambda} + \frac{1}{2} \frac{G_R^2}{H_R + \lambda} - \frac{1}{2} \frac{(G_L + G_R)^2}{H_L + H_R + \lambda} - \gamma$ ,
11:  Update the model:  $f_t(x) = f_{t-1}(x) + \eta \cdot F_t(x)$ 
12: end for
13: Output: Trained FRDT model  $f_T(x)$ 

```

normal behavior [17], [18], [20]. Building on this idea, we propose a feature engineering method, termed pattern feature, to model contextual dependencies in log event sequences. By explicitly indicating localized contextual deviations, pattern features provide a compact and informative representation for downstream anomaly detection tasks.

The pattern feature method produces a binary vector that highlights potential anomalies at each position within an event sequence by leveraging a trained pattern model. To construct this vector, each event in the input sequence is sequentially masked, and the model is tasked with predicting the masked event based on its surrounding context. If the predicted event matches the original event, the corresponding feature value is assigned 1, indicating consistency with the learned normal behavior. Conversely, if the prediction fails to match the actual event, the feature value is set to 0, suggesting a potential anomaly. Once all events have been processed, the resulting binary values are assembled in their original sequence order to form the final pattern feature vector. The generation process of the pattern feature is illustrated in Fig. 6, and the training procedure for the pattern model is described in Algorithm 4.

The symbol $\mathbb{I}(\bullet)$ is the indicator function, if $\bullet$ is true then the result is 1, otherwise 0. The function $mask(x, j)$ creates a vector by masking j -th event of the sequence x . Lines 3-7 of the algorithm describe the preparation of the training dataset for the pattern model f_m , which is an STree model. In line 2, the duplicate sequence samples are removed from the dataset. It allows the pattern model to focus on learning sequence patterns during training without being overly influenced by sample distribution. In line 5, the masked sequence $mask(x, j)$ is incorporated as a feature and the event k_j as a label, both of which are appended to the dataset D' designated for the training of the pattern model. Lines 10-15 delineate the process of generating pattern features for all samples within the dataset. For each sample, a pattern feature vector $z = (z_1, z_2, \dots, z_n)$ is generated.

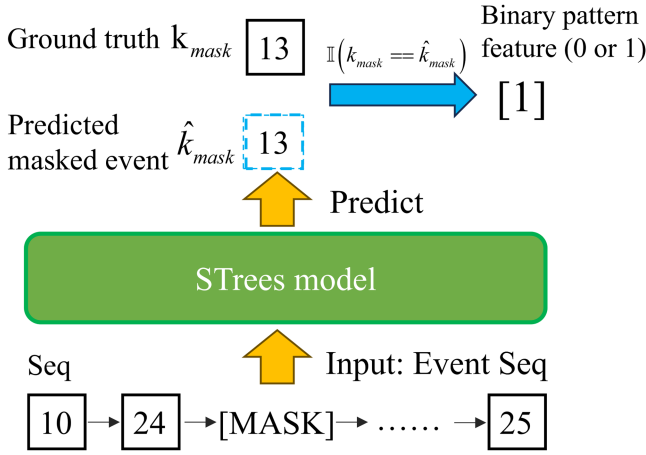


Fig. 6. Generation of the pattern feature based on masked event prediction.

Algorithm 4: Pattern Model and Pattern Feature.

- 1: **Input:** Training sequence dataset $D = \{(\mathbf{x}_i, y_i)\}_{i=1}^I$, the length of event sequence n
 - 2: Let $D' = \emptyset$, $D_u = \text{remove_duplicate}(D)$
 - 3: **for** $\mathbf{x}$ in D_u **do**
 - 4: **for** $j = 1, 2, \dots, n$ **do**
 - 5: $D' = D' \cup (\text{mask}(\mathbf{x}, j), k_j)$
 - 6: **end for**
 - 7: **end for**
 - 8: Train pattern model f_m using mask dataset D'
 - 9: $R = \emptyset$
 - 10: **for** $\mathbf{x}$ in D **do**
 - 11: **for** $j = 1, 2, \dots, n$ **do**
 - 12: $z_j = \mathbb{I}(f_m(\text{mask}(\mathbf{x}, j)) == k_j)$
 - 13: **end for**
 - 14: $R = R \cup (z_1, z_2, \dots, z_n)$
 - 15: **end for**
 - 16: **Output:** Pattern feature dataset R , trained pattern model f_m
-

Integrating pattern features with original sequence features allows the anomaly detection model to retain complementary information from both sources, enabling it to learn more comprehensive representations and improve its ability to distinguish between normal and anomalous samples. This combined feature is referred to as the fusion feature, as described in (5),

$$\mathbf{x}_{\text{fuse}} = \mathbf{x} \oplus \mathbf{z}, \quad (5)$$

where $\oplus$ denotes vector concatenation.

D. Log Anomaly Detection Model

To construct the anomaly detection model, we leverage both the pattern feature and the NCL method. Noise samples are generated following Algorithm 1, where normal sequences are labeled as 1 and noise sequences as 0. Pattern feature vectors are generated using Algorithm 4, which mark potential anomalies

Algorithm 5: Log Anomaly Detection Model Training.

- 1: **Input:** Training normal sequence dataset $D_{\text{normal}} = \{(\mathbf{x}_i, y_i)\}_{i=1}^I$, sample size $J \leq I$, number of noise events e , and ratio K of noise samples to normal samples
- 2: Train the pattern model f_m using D_{normal} and Algorithm 4
- 3: Randomly choose J samples from D_{normal} to construct a subset D'_{normal} for training the detection model.
- 4: Generate KI noise samples to form D_{noise} from D'_{normal} using Algorithm 1 with e noise events
- 5: Merge D'_{normal} (as positive samples) and D_{noise} (as negative samples) to form the training dataset D_{train}
- 6: Generate pattern feature dataset R_{train} based on D_{train} using f_m and Algorithm 4
- 7: Concatenate sequence features D_{train} and pattern features R_{train} to form the fusion feature dataset D'_{train} :

$$D'_{\text{train}} = D_{\text{train}} \oplus R_{\text{train}}$$

- 8: Train the log anomaly detection model g_θ on D'_{train} using STrees and the NCL loss defined in (3)
 - 9: **Output:** Trained log anomaly detection model g_θ
-

within normal sequences and help the classifier identify deviations more effectively. These features are concatenated with the original event sequence features to form the fusion feature, as defined in (5). The fusion feature serves as the input to the STrees classifier, which is trained by the NCL method to learn a discriminative decision boundary. The complete training procedure of the proposed log anomaly detection model is outlined in Algorithm 5.

IV. PERFORMANCE EVALUATION

A. Experimental Settings

In the experiments, we used four real-world system log datasets: HDFS [41], BGL [42], OpenStack [20], [43], and Hadoop [44]. The details for each dataset are as follows.

HDFS: The dataset was generated by executing Hadoop-based map-reduce tasks across over 200 nodes on Amazon's EC2 platform, with labeling conducted by experts in the Hadoop domain. Out of a total of 111,979,954 log entries compiled, approximately 2.9% were classified as anomaly samples, encompassing incidents like “write exceptions” [41].

BGL: The BGL dataset comprised 4,747,963 logs. Every log within the BGL dataset has been manually categorized as either anomalous or normal, with 348,460 logs identified as anomalous. This dataset was produced by the Blue Gene/L supercomputer, which featured 128,000 processors and was situated at the Lawrence Livermore National Laboratory (LLNL) [42].

OpenStack: An OpenStack experiment, utilizing the Mitaka version, was conducted on CloudLab, comprising one control node, one network node, and eight compute nodes. From a

TABLE III
EVENT SEQUENCE STATISTICS FOR SYSTEM LOG DATASETS

Dataset	Training (normal)	Test data (normal)	(anomaly)	# Log keys
HDFS	8,000	5,258,554	168,888	29
BGL	32,000	2,973,436	1,110,955	393
OpenStack	8,000	33,247	3,524	40
Hadoop	8,000	21,536	360,046	349

total of 1,335,318 log entries gathered, approximately 7% were identified as abnormal. A script was continuously operated to perform virtual machine (VM)-related activities, such as creation and deletion of VMs, stopping and starting, pausing and unpausing, as well as suspending and resuming [20], [43].

Hadoop: The dataset originated from a 46-core Hadoop cluster on five i7-3770 CPU, 16 GB RAM machines, featuring logs from WordCount and PageRank applications. It included scenarios of normal operations and intentional failures like machine shutdown, network disconnection, and disk full to simulate real-world anomalies [44].

We employ the LogParser tool [21], [45] to preprocess the raw log data into log event sequences. Specifically, following the same preprocessing pipeline as DeepLog [20], we utilize the Drain algorithm [46] to extract log templates and map raw textual logs into sequences of discrete event IDs, thereby forming standardized event sequences. Subsequently, a sliding window mechanism is applied to segment the event sequences, with the window size set to 10 [20]. Sequences with lengths shorter than 10 are discarded.

The number of training and testing samples for each log dataset is listed in Table III. Identical event sequences may occur in both normal and anomalous samples, potentially leading to label inconsistencies that hinder model learning. To address this, we retain such sequences from the normal set and remove their duplicates from the anomalous set, ensuring that each training sequence has a unique and consistent label. During model training, only normal samples are used, while the test sets contain both normal and anomalous samples for evaluating anomaly detection performance.

The ratio K of noise samples to normal samples is 1. The sample size J in Algorithm 5 for training the log anomaly detection model is set to 1000. In STrees, FRDT and Ex-Tree are implemented using the LightGBM library [38] and the ExtraTreesClassifier from the Scikit-learn library [47] in Python, respectively. Both models use 100 trees. The learning rate for FRDT is set to 0.01, while for ExTree, the parameter $max_features$ is set to 0.1 and the criterion is set to “entropy”.

B. Evaluation Metrics

The anomaly detection task is a binary classification problem. Due to the significant difference in sample sizes between the normal and anomaly classes, we primarily focus on the recall of the normal class and the anomaly class. In this paper, we treat the normal class as the positive class, and the anomaly class as the negative class. The definitions of normal-class recall and

anomaly-class recall are as follows:

$$\text{Normal-class recall} = \frac{TP}{TP + FN}, \quad (6)$$

$$\text{Anomaly-class recall} = \frac{TN}{TN + FP}, \quad (7)$$

where:

- True Positive (TP) is the number of normal samples correctly classified as normal (positive class);
- False Negative (FN) is the number of normal samples misclassified as anomaly (negative class);
- True Negative (TN) is the number of anomaly samples correctly classified as anomaly (negative class);
- False Positive (FP) is the number of anomaly samples misclassified as normal (positive class).

C. Model Comparison on Log Anomaly Detection

To evaluate the anomaly detection capability of our proposed model, we compared it with LogDeep and DeepAD methods. All models were evaluated using normal-class and anomaly-class recall as the metrics. In our proposed model, fusion features were used as input features, and the number of synthesized noise events was set to 2, 2, and 1 for the HDFS, BGL, and Hadoop datasets, respectively. The evaluation results are shown in Table IV.

On the HDFS dataset, our model demonstrates at least a 0.97% improvement in normal-class recall and a 14.48% improvement in anomaly-class recall compared to LogDeep methods. Among all DeepAD approaches, the ICL method achieves the highest recall for both normal and anomaly classes. However, while ICL surpasses our model by 5.74% in normal-class recall, it exhibits a 23.32% lower recall in detecting anomalies.

On the BGL dataset, our model achieves a superior performance in normal-class recall compared to all LogDeep methods, while also exhibiting at least a 5.3% improvement in anomaly-class recall. Although DeepAD methods surpass our model by 3% to 5% in the normal-class recall, its anomaly-class recall is at least 45% lower.

On the Hadoop dataset, our method achieves at least 9.29% higher anomaly-class recall compared to LogDeep methods, while surpassing all methods except AE-IForest in normal-class recall. The AE-IForest model has a 6.51% higher normal-class recall compared to our model, but it has a 10.79% lower anomaly-class recall. Although the anomaly-class recall of all DeepAD methods exceeds 90%, the highest among them is 4.15% higher than our model. However, its highest normal-class recall is 53.19%, which is 29.68% lower than our model.

In the comparative analysis, we observed a significant bias in the performance of DeepAD methods when dealing with different datasets. Specifically, the DeepAD methods tend to prioritize the recognition of the normal class on the HDFS and BGL datasets, while it leans towards the anomaly class on the Hadoop dataset. This preference results in achieving higher detection recall in its preferred class, while the detection performance in the opposite class is significantly poorer. In contrast, LogDeep methods consistently exhibit a bias towards

TABLE IV
RECALL OF ANOMALY DETECTION MODELS ON HDFS, BGL, AND HADOOP

Dataset	HDFS		BGL		Hadoop	
Recall (%)	Normal	Anomaly	Normal	Anomaly	Normal	Anomaly
LogDeep methods						
DeepLog [20]	88.35	58.76	87.61	69.49	82.16	31.65
Tiesias [48]	90.58	52.47	88.67	66.46	87.40	28.81
DEEPCASE [16]	90.52	51.74	88.49	60.79	84.62	29.55
LogAnomaly [17]	90.97	53.61	88.59	67.45	86.36	29.20
AE-IForest [49]	83.52	47.40	87.51	33.53	89.38	79.77
LogBERT [18]	89.85	75.92	89.68	79.92	80.49	62.60
LAnoBERT [19]	82.31	53.17	89.88	80.41	80.59	81.27
DeepAD methods						
DEEPSVDD [13]	96.95	56.42	94.29	37.49	15.45	90.82
REPEN [32]	94.44	22.02	93.60	14.47	0.91	90.89
RDP [31]	96.30	47.12	95.11	34.60	53.19	94.71
RCA [30]	95.25	32.10	94.51	26.60	52.93	94.68
NeuTraL [27]	97.40	62.94	93.53	14.55	9.51	90.76
ICL [29]	97.68	67.08	92.96	17.25	28.52	92.15
DeepIForest [14]	95.34	33.38	94.01	20.74	0.53	90.13
Our method						
STrees	91.94	90.40	89.95	85.71	82.87	90.56

TABLE V
ACCURACY AND F1 SCORE OF 1-STEP PREDICTION

Dataset	HDFS		BGL		OpenStack		Hadoop	
Metric (%)	ACC	F1	ACC	F1	ACC	F1	ACC	F1
LogDeep methods								
DeepLog [20]	90.71	90.00	88.70	87.98	92.56	92.59	84.25	81.67
Tiesias [48]	90.89	90.62	88.69	87.85	93.26	93.20	87.19	85.97
DEEPCASE [16]	90.56	89.95	88.70	87.88	80.36	79.80	83.51	80.63
LogAnomaly [17]	90.99	90.41	88.72	87.92	91.90	91.78	87.06	85.16
LogBERT [18]	85.50	81.31	80.42	73.98	96.89	97.14	76.36	68.81
Our method								
STrees	91.30	90.96	89.02	88.40	98.84	98.82	89.93	89.22

the normal class, leading to higher recall for the normal class compared to the anomaly class.

When evaluating our method, the results indicate that our model achieves higher normal-class and anomaly-class recall compared to the LogDeep methods on the HDFS and BGL datasets. On the Hadoop dataset, our model achieves the highest recall in the anomaly class compared to the LogDeep methods. Compared to DeepAD methods, the maximum recall gap for our model in the preferred class does not exceed 6%. However, for the non-preferred class, our model achieves a recall that is at least 23% higher. These experimental results demonstrate that our model exhibits superior performance compared to LogDeep methods on most datasets, and a more balanced detection capability compared to the DeepAD methods.

D. Evaluation of STrees for Learning Log Event Sequence Context

To evaluate STrees' ability to capture contextual patterns and event relationships in log event sequences, we designed a

predictive experiment focusing on both short-term and long-term sequence modeling. Specifically, we evaluated the model using two types of next-event prediction tasks: 1-step and N-step prediction, which measure its ability to understand event correlations across varying temporal spans. In this experiment, STrees was used as the prediction model, with only event sequence as input features. To ensure the model learns only the typical system behavior, both training and testing were conducted exclusively on normal log sequences across all four datasets: HDFS, BGL, OpenStack, and Hadoop. The evaluation results are presented in Table V and Fig. 7.

For the 1-step prediction task, the STrees model demonstrated its ability to accurately predict immediate future events. Across the four datasets, we observe minimal performance differences among the models in LogDeep methods. This suggests that the model performance on these datasets may have approached an upper bound in terms of prediction accuracy. The STrees model achieves the highest prediction accuracy and F1 scores, demonstrating its excellent capability in event prediction tasks. Compared to LogDeep methods, the STrees model shows an

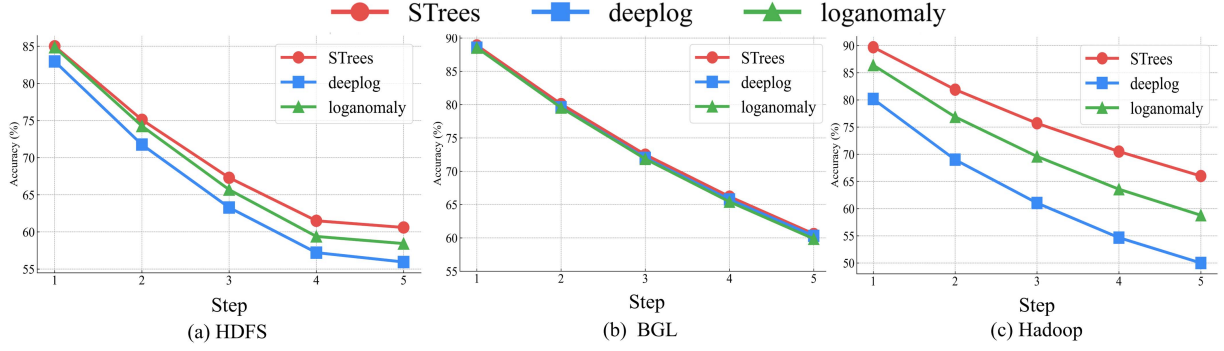


Fig. 7. Accuracy of N-step prediction.

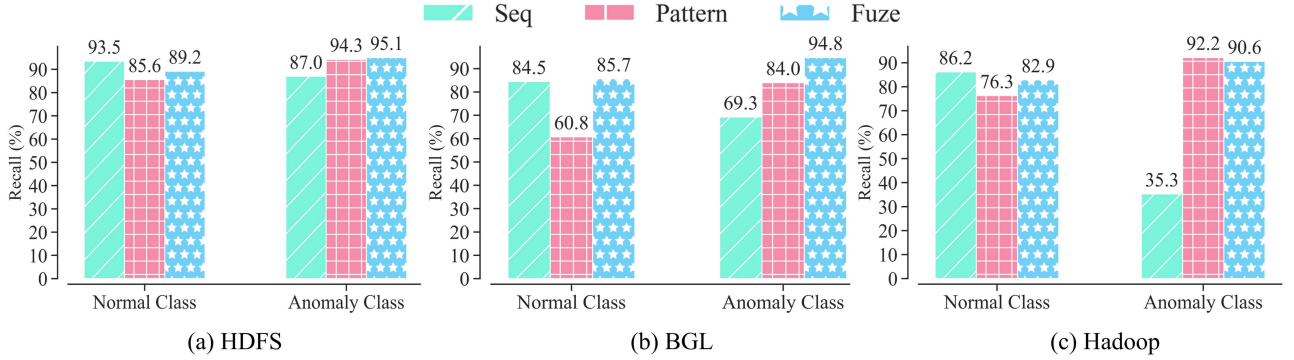


Fig. 8. Recall performance of three features on different datasets.

accuracy improvement ranging from 0.3% to 2.7% across all datasets.

For the N-step prediction task, we evaluated the robustness of the STrees model in long-term event predictions, assessing its performance as the prediction horizon increased. From step 1 to step 5, the accuracy of the STrees model decreases by 24.4%, 28.3%, and 23.7% on the HDFS, BGL, and Hadoop datasets, respectively. This decline is expected, as errors in the prefix of the generated event sequence may propagate and cause subsequent predictions to deviate from the true distribution. Among the compared models, STrees consistently achieves the highest prediction accuracy across all steps. As the prediction step increases, the accuracy gap between models also gradually widens. These experimental results demonstrate that the long-term prediction ability of the models significantly decreases with an increase in the number of predicted event steps, and the performance differences between the models also slightly increase accordingly.

Overall, the results from both short-term and long-term prediction tasks demonstrate STrees' strong ability to capture contextual dependencies and event relationships in log event sequences, leading to superior predictive performance across various scenarios.

E. Assessing the Impact of the Pattern Feature on Model Recall Performance

To evaluate the impact of pattern features on the performance of anomaly detection models, we conducted experiments

comparing three different feature configurations: sequence features (Seq), pattern features (Pattern), and fusion features (Fuze) on the recall performance of the models. The sequence feature refers to directly using the event sequence as input features. The pattern feature is derived from the event sequence based on Algorithm 4. The fusion feature is obtained by concatenating the event sequence and pattern features, as defined in (5). The number of noise events e is set to 1. The experimental results are presented in Fig. 8.

The experiment shows that sequence features excel in detecting normal samples, while pattern features are more effective for anomalous samples. For example, on the BGL dataset, pattern features show a decrease of 23.7% in normal-class recall compared to sequence features. On the Hadoop dataset, sequence features have a 56.9% lower normal-class recall compared to pattern features. Despite achieving high recall for specific classes, i.e., normal or anomaly class, both sequence features and pattern features perform poorly in identifying the opposite class.

Fusion features effectively alleviate the tendency of sequence features and pattern features to lean towards a specific class in anomaly detection tasks. Experimental results on the HDFS dataset demonstrate that the normal-class recall of the fusion feature is slightly lower than that of the sequence feature by 4.3%, but it achieves the highest anomaly-class recall. On the BGL dataset, fusion features outperform single features not only in the normal class but also in the anomaly class, demonstrating their superior overall performance. On the Hadoop dataset, the normal-class recall of fusion features is 3.3% lower than using sequence features, and it is 6.6% higher than that of pattern

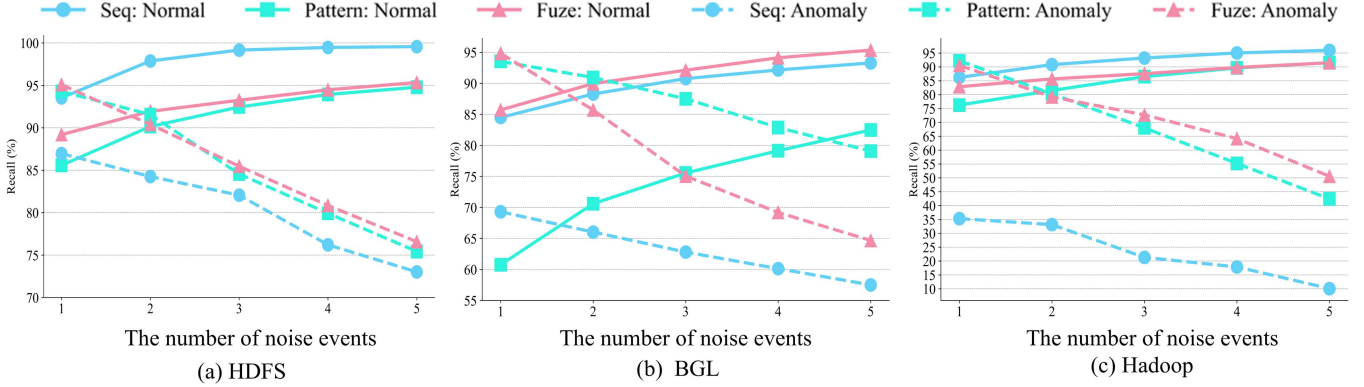


Fig. 9. Impact of the number of noise events on recall performance across different datasets.

features. In the anomaly class, it is 55.3% higher than sequence features, and only 1.6% lower than pattern features.

These experimental results indicate that pattern features effectively enhance the model's ability to identify anomaly class samples. However, pattern features may also degrade the performance on normal-class samples. In contrast, fusion features mitigate the tendency of sequence features and pattern features to lean towards a specific class in anomaly detection tasks by integrating sequence and pattern features. Fusion features not only improve the model's ability to identify anomaly class samples but also maintain robust identification performance for normal class samples.

F. Assessing the Impact of Noise Events on Model Recall Performance

We explored the impact of varying the number of noise events e on the anomaly detection model performance. The experiment utilized sequence features, pattern features, and fusion features. The experimental results are shown in Fig. 9.

We observe that increasing the number of noise events leads to a gradual decrease in anomaly-class recall and a corresponding increase in normal-class recall across all three feature types. When the number of noise events is small and noisy samples closely resemble normal samples, the anomaly detection classifier requires greater discriminative capacity to distinguish between them, resulting in a tighter decision boundary. Consequently, the model becomes highly sensitive to slight deviations from the normal class, which improves its ability to detect anomalies but also increases the risk of misclassifying borderline normal instances. This results in a relatively higher anomaly-class recall and a lower normal-class recall. As the number of noise events increases, the separation between noisy and normal samples becomes more distinct, enabling the anomaly detection classifier to adopt a looser decision boundary. This looser decision boundary improves the recall for the normal class while reducing the recall for the anomaly class.

Overall, the number of noise events functions as a sensitivity parameter that influences the model's preference toward detecting either subtle or severe anomalies. This parameter also affects the trade-off between false positives and false negatives.

TABLE VI
COMPARISON OF INFERENCE TIME ON 500,000 LOG SEQUENCES

Model	Total Time (s)	Avg. Time per Sample (s)
STrees (ours)	11.42	2.28×10^{-5}
DeepLog	13.98	2.80×10^{-5}
LogBERT	15.04	3.01×10^{-5}
LogAnomaly	60.91	1.22×10^{-4}

Specifically, smaller e leads to higher sensitivity, enabling the detection of subtle anomalies but at the expense of increased false alarms; conversely, larger e leads to lower sensitivity, reducing false alarms while potentially missing milder anomalies.

G. Computational Efficiency Evaluation

To assess the computational cost during inference, we measured the time required for various models to perform predictions on a large, representative test set. The test set consisted of 500,000 pre-processed log sequences. All experiments were conducted on a server equipped with a 64-core Intel(R) Xeon(R) Gold 6242 CPU. For each model, we recorded the total time taken to process the entire data and computed the average inference time per log sequence. We focus on inference efficiency rather than training cost, as model inference directly affects system throughput during deployment, while training is performed offline and has limited impact on runtime performance. This comparison includes our proposed STrees model alongside DeepLog, LogBERT, and LogAnomaly. The inference time results are shown in Table VI.

STrees is at least 18.3% faster than DeepLog and LogBERT. It achieves the lowest total processing time on 500,000 samples, resulting in the fastest average inference time per sample. These results highlight the superior computational efficiency of STrees during CPU-based inference.

V. CONCLUSION

In this paper, we propose the NCL method for log anomaly detection. Unlike previous approaches that rely on event prediction as an intermediate training task, NCL directly determines whether a log event sequence is anomalous. This design avoids

the objective mismatch introduced by the auxiliary prediction task. We also introduce the STrees model and the pattern feature, both of which effectively model contextual dependencies within log event sequences. Specifically, the pattern feature highlights potentially anomalous events, which helps the model better identify anomaly-class samples. Experimental results show that our approach outperforms LogDeep methods in most comparisons while maintaining a more balanced recall compared to DeepAD methods. These findings suggest that the model can establish more robust decision boundaries for anomaly detection.

In future work, we aim to enhance STrees along two technical directions. First, we will investigate knowledge distillation to compress the ensemble into a compact student model that maintains accuracy while significantly reducing inference costs. Second, we plan to apply ensemble pruning to eliminate redundant components, thereby improving computational efficiency and reducing memory usage. Furthermore, to extend our model to longer event sequences, we will explore dimensionality reduction techniques such as principal component analysis and feature selection methods. While these techniques are promising for improving scalability, effectively representing long sequences requires deeper analysis to capture underlying patterns and event dependencies. These efforts are expected to improve the scalability of our model for large-scale log anomaly detection without compromising performance.

REFERENCES

- [1] Y. Liu, Y. Gu, X. Shen, Q. Liao, and Q. Yu, "MSCA: An unsupervised anomaly detection system for network security in backbone network," *IEEE Trans. Netw. Sci. Eng.*, vol. 10, no. 1, pp. 223–238, Jan./Feb. 2023.
- [2] S. Xu, Y. Qian, and R. Q. Hu, "Data-driven edge intelligence for robust network anomaly detection," *IEEE Trans. Netw. Sci. Eng.*, vol. 7, no. 3, pp. 1481–1492, Jul.–Sep. 2020.
- [3] C.-J. Chew, W.-B. Lee, T.-H. Chen, I.-C. Lin, and J.-S. Lee, "Log preservation in custody dual blockchain with energy regime and obfuscation shuffle," *IEEE Trans. Netw. Sci. Eng.*, vol. 11, no. 4, pp. 3495–3511, Jul./Aug. 2024.
- [4] Z. Lian, P. Shi, C. P. Lim, I. J. Radas, and R. K. Agarwal, "Hybrid stealthy attacks on stochastic event-based remote estimation under packet dropouts," *IEEE Trans. Netw. Sci. Eng.*, vol. 11, no. 6, pp. 5829–5838, Nov./Dec. 2024.
- [5] Z. Zhang, J. Liu, S. Zhang, and H. Zhang, "Event-based fault detection for networked switched systems subject to hybrid attacks," *IEEE Trans. Netw. Sci. Eng.*, vol. 11, no. 3, pp. 2937–2950, May/Jun. 2024.
- [6] Y. Zhang, Z. Wang, L. Zou, H. Dong, and X. Yi, "Neural-network-based secure state estimation under energy-constrained denial-of-service attacks: An encoding-decoding scheme," *IEEE Trans. Netw. Sci. Eng.*, vol. 10, no. 4, pp. 2002–2015, Jul./Aug. 2023.
- [7] S. Gao, H. Zhang, Z. Wang, C. Huang, and H. Yan, "Data-driven injection attack strategy for linear cyber-physical systems: An input-output data-based approach," *IEEE Trans. Netw. Sci. Eng.*, vol. 10, no. 6, pp. 4082–4095, Nov./Dec. 2023.
- [8] W. Zhan, Z. Miao, Y. Chen, Z.-G. Wu, and Y. Wang, "Event-triggered finite-time formation control for networked nonholonomic mobile robots under denial-of-service attacks," *IEEE Trans. Netw. Sci. Eng.*, vol. 10, no. 6, pp. 3754–3766, Nov./Dec. 2023.
- [9] C. Almodovar, F. Sabrina, S. Karimi, and S. Azad, "LogFiT: Log anomaly detection using fine-tuned language models," *IEEE Trans. Netw. Service Manag.*, vol. 21, no. 2, pp. 1715–1723, Apr. 2024.
- [10] Z. Wu, H. Li, Y. Qian, Y. Hua, and H. Gan, "Poison-resilient anomaly detection: Mitigating poisoning attacks in semi-supervised encrypted traffic anomaly detection," *IEEE Trans. Netw. Sci. Eng.*, vol. 11, no. 5, pp. 4744–4757, Sep./Oct. 2024.
- [11] X. Ma and W. Shi, "AESMOTE: Adversarial reinforcement learning with SMOTE for anomaly detection," *IEEE Trans. Netw. Sci. Eng.*, vol. 8, no. 2, pp. 943–956, Apr.–Jun. 2021.
- [12] J. Qi et al., "LogEncoder: Log-based contrastive representation learning for anomaly detection," *IEEE Trans. Netw. Service Manag.*, vol. 20, no. 2, pp. 1378–1391, Jun. 2023.
- [13] L. Ruff et al., "Deep one-class classification," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 4393–4402.
- [14] H. Xu, G. Pang, Y. Wang, and Y. Wang, "Deep isolation forest for anomaly detection," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 12, pp. 12591–12604, Dec. 2023.
- [15] V.-H. Le and H. Zhang, "Log-based anomaly detection with deep learning: How far are we?," in *Proc. 44th Int. Conf. Softw. Eng.*, 2022, pp. 1356–1367.
- [16] T. Van Ede et al., "DEEPCASE: Semi-supervised contextual analysis of security events," in *Proc. IEEE Symp. Secur. Privacy (SP)*, 2022, pp. 522–539.
- [17] W. Meng et al., "Loganomaly: Unsupervised detection of sequential and quantitative anomalies in unstructured logs," in *Proc. Int. Joint Conf. Artif. Intell.*, 2019, pp. 4739–4745.
- [18] H. Guo, S. Yuan, and X. Wu, "LogBERT: Log anomaly detection via BERT," in *Proc. 2021 Int. Joint Conf. Neural Netw.*, 2021, pp. 1–8.
- [19] Y. Lee, J. Kim, and P. Kang, "LanoBERT: System log anomaly detection based on BERT masked language model," *Appl. Soft Comput.*, vol. 146, 2023, Art. no. 110689.
- [20] M. Du, F. Li, G. Zheng, and V. Srikumar, "DeepLog: Anomaly detection and diagnosis from system logs through deep learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2017, pp. 1285–1298.
- [21] P. He, J. Zhu, S. He, J. Li, and M. R. Lyu, "An evaluation study on log parsing and its use in log mining," in *Proc. 46th Annu. IEEE/IFIP Int. Conf. Dependable Syst. Netw.*, 2016, pp. 654–661.
- [22] A. Vaswani et al., "Attention is all you need," in *Proc. 31st Int. Conf. Neural Inf. Process. Syst.*, 2017, pp. 6000–6010.
- [23] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. conf. North Amer. chapter assoc. computat. linguistics: human language technologies*, vol. 1, pp. 4171–4186, 2019.
- [24] X. Zhang et al., "Robust log-based anomaly detection on unstable log data," in *Proc. 27th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, 2019, pp. 807–817.
- [25] D. M. Tax and R. P. Duin, "Support vector data description," *Mach. Learn.*, vol. 54, pp. 45–66, 2004.
- [26] L. Bergman and Y. Hoshen, "Classification-based anomaly detection for general data," *8th Int. Conf. Learn. Representations, ICLR 2020*, 2020.
- [27] C. Qiu, T. Pfommer, M. Kloft, S. Mandt, and M. Rudolph, "Neural transformation learning for deep anomaly detection beyond images," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 8703–8714.
- [28] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proc. 8th IEEE Int. Conf. Data Mining*, 2008, pp. 413–422.
- [29] T. Shenkar and L. Wolf, "Anomaly detection for tabular data with internal contrastive learning," in *Proc. 10th Int. Conf. Learn. Representations*, 2022.
- [30] B. Liu, D. Wang, K. Lin, P.-N. Tan, and J. Zhou, "RCA: A deep collaborative autoencoder approach for anomaly detection," in *Proc. Conf. Artif. Intell.*, NIH Public Access, 2021, pp. 1505–1511.
- [31] H. Wang, G. Pang, C. Shen, and C. Ma, "Unsupervised representation learning by predicting random distances," *IJCAI Int. Joint Conf. Artif. Intell.*, pp. 2950–2956, 2019, *arXiv:1912.12186*.
- [32] G. Pang, L. Cao, L. Chen, and H. Liu, "Learning representations of ultrahigh-dimensional data for random distance-based outlier detection," in *Proc. 24th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2018, pp. 2041–2050.
- [33] M. Gutmann and A. Hyvärinen, "Noise-contrastive estimation: A new estimation principle for unnormalized statistical models," in *Proc. 13th Int. Conf. Artif. Intell. Statist.*, 2010, pp. 297–304.
- [34] X. He et al., "Practical lessons from predicting clicks on ads at Facebook," in *Proc. 8th Int. Workshop Data Mining Online Advertising*, 2014, pp. 1–9.
- [35] P. Geurts, D. Ernst, and L. Wehenkel, "Extremely randomized trees," *Mach. Learn.*, vol. 63, pp. 3–42, 2006.
- [36] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, pp. 5–32, 2001.
- [37] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2016, pp. 785–794.
- [38] G. Ke et al., "LightGBM: A highly efficient gradient boosting decision tree," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 3149–3157.

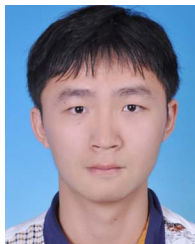
- [39] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *Ann. Statist.*, vol. 29, pp. 1189–1232, 2001.
- [40] A. Dempster, D. F. Schmidt, and G. I. Webb, "QUANT: A minimalist interval method for time series classification," *Data Mining Knowl. Discov.*, vol. 38, no. 4, pp. 2377–2402, 2024.
- [41] W. Xu, L. Huang, A. Fox, D. Patterson, and M. I. Jordan, "Detecting large-scale system problems by mining console logs," in *Proc. ACM SIGOPS 22nd Symp. Operating Syst. Princ.*, 2009, pp. 117–132.
- [42] A. Oliner and J. Stearley, "What supercomputers say: A study of five system logs," in *Proc. 37th Annu. IEEE/IFIP Int. Conf. Dependable Syst. Netw.*, 2007, pp. 575–584.
- [43] S. He, J. Zhu, P. He, and M. R. Lyu, "Loghub: A large collection of system log datasets towards automated log analytics," *2023 IEEE 34th Int. Symp. Softw. Rel. Eng. (ISSRE)*, 2020, pp. 355–366.
- [44] Q. Lin, H. Zhang, J.-G. Lou, Y. Zhang, and X. Chen, "Log clustering based problem identification for online service systems," in *Proc. 38th Int. Conf. Softw. Eng. Companion*, 2016, pp. 102–111.
- [45] J. Zhu et al., "Tools and benchmarks for automated log parsing," in *Proc. IEEE/ACM 41st Int. Conf. Softw. Eng.: Softw. Eng. Pract.*, 2019, pp. 121–130.
- [46] P. He, J. Zhu, Z. Zheng, and M. R. Lyu, "Drain: An online log parsing approach with fixed depth tree," in *Proc. IEEE Int. Conf. Web Serv.*, 2017, pp. 33–40.
- [47] F. Pedregosa et al., "Scikit-Learn: Machine learning in python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.
- [48] Y. Shen, E. Mariconti, P. A. Vervier, and G. Stringhini, "Tiresias: Predicting security events through deep learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2018, pp. 592–605.
- [49] A. Farzad and T. A. Gulliver, "Unsupervised log message anomaly detection," *ICT Exp.*, vol. 6, no. 3, pp. 229–237, 2020.



Genke Yang received the B.S. degree in mathematics from Shanxi University, Taiyuan, China, in 1984, the M.S. degree in mathematics from Xinan Normal University, China, in 1987, and the Ph.D. degree in systems engineering from Xi'an Jiaotong University, Xi'an, China, in 1998. He is currently a Full Professor with the Department of Automation, Shanghai Jiao Tong University, Shanghai, China. His research interests include industrial control system security and industrial process control.



Jian Chu received the B.S., M.S., and Ph.D. degrees from Zhejiang University, Hangzhou, China, in 1982, 1984, and 1989, respectively, and the Ph.D. degree from Joint Education Program, Zhejiang University, Hangzhou, China, and Kyoto University, Kyoto, Japan. He is currently a Full Professor with the Department of Automation, Shanghai Jiao Tong University, Shanghai, China. His research interests include cyber system control, advanced process control, and industrial control system security.



Pengcheng Luo received the B.S. degree in mechanical engineering and automation from the East China University of Science and Technology, Shanghai, China, in 2018, the M.S. degree in mechanical engineering from Tongji University, Shanghai, China, in 2021. Since 2022, he has been working toward Ph.D. degree with Shanghai Jiao Tong University, Shanghai, China. His research interests include modelling and analyzing of network communication with machine learning techniques.



Boon-Hee Soong (Senior Member, IEEE) received the B.Eng. (with Hons.) degree in electrical and electronic engineering from the University of Auckland, Auckland, New Zealand, in 1984, and the Ph.D. degree in telecommunication from the University of Newcastle, Callaghan, NSW, Australia, in 1990. From 1999 to 2000, he was a Visiting Research Fellow with the Department of Electrical and Electronic Engineering, Imperial College, London, U.K., under the Commonwealth Fellowship Award. He is currently an Associate Professor with the School of Electrical and Electronic Engineering, Nanyang Technological University, Singapore.



Dengke Deng received the B.S. degree from Harbin Engineering University, Harbin, China, in 2022. He is currently working toward the M.S. degree with Shanghai Jiao Tong University, Shanghai, China. His research interests include control system security and natural language processing.



Mingfeng Xie received the B.S. degree in electronic information engineering from the Nanjing University of Information Science and Technology, Nanjing, China, in 2019. He is currently working toward the Ph.D. degree with the College of Electronic and Information Engineering, Nanjing University of Aeronautics and Astronautics, Nanjing. His research interests include near-field communications, cell-free massive MIMO, and performance analysis in wireless communications.



Chau Yuen (Fellow, IEEE) received the B.Eng. and Ph.D. degrees from Nanyang Technological University, Singapore, in 2000 and 2004, respectively. In 2005, he was a Postdoctoral Fellow with the Lucent Technologies Bell Laboratories, Murray Hill, NJ, USA. From 2006 to 2010, he was with the Institute for Infocomm Research, Singapore. From 2010 to 2023, he was with the Engineering Product Development Pillar, Singapore University of Technology and Design. Since 2023, he has been with the School of Electrical and Electronic Engineering, Nanyang Technological University, as an Associate Professor. He is currently a Distinguished Lecturer of the IEEE Vehicular Technology Society, a top 2% Scientist by Stanford University.